nabia: Architecture and Methodology

A systematic trend strategy platform: C++ core, Python layer, AI agents and web app

nabia documentation

October 2026

Abstract

This document describes the whole nabia project: its purpose, its architecture, and the methodology of every component. That covers the C++ numerical core (time series, signal filters, forecasters, optimizers, risk measures, the walk-forward risk budgeting engine), the command-line application, the Python bindings, the market data pipeline, the three AI agents with their MCP tool servers, and the web application that operates them. Each method is stated precisely, and the code that implements it is shown with excerpts *extracted from the source at build time*, each labelled with its file and line numbers, so the document stays in step with the code. A dedicated section explains how the sentiment and relevance of news articles are produced and aggregated, and what that implies for their use. The individual algorithms of the forecasting and convex optimization modules are derived in detail, with proofs, in the companion documents of `doc/forecast` and `doc/optimization`.

Contents

1 Purpose and overview **3**

2 Architecture **3**
- 2.1 Layers . . . 4
- 2.2 Directory map . . . 4
- 2.3 Build and dependencies . . . 4

3 Time series containers **4**

4 Signal extraction **5**

5 Forecasting **6**

6 Optimization **6**

7 Risk measures and risk budgeting **8**
- 7.1 Risk measures . . . 8
- 7.2 Risk budgeting . . . 8

8 The walk-forward portfolio engine **9**
- 8.1 Timing: no look-ahead . . . 9
- 8.2 Covariance estimation . . . 9
- 8.3 Allocation and volatility targeting . . . 10
- 8.4 Parallel solves, sequential accounting . . . 10
- 8.5 Drift, trading costs and accounting . . . 11

9 Performance metrics **12**

10 Command-line application and Python bindings **13**

11 Market data **14**
11.1 Universe and sources 14
11.2 Storage and incremental updates 15

12 AI agents **16**
12.1 Roles and data flow 16
12.2 MCP servers and point-in-time tools 17
12.3 The agent loop 18
12.4 Statistics computed for the stats analyzer 19
12.5 From decisions to positions 21
12.6 The trend rule baseline 22
12.7 Backtest mode 22

13 News: collection, sentiment and relevance **23**
13.1 Collection 23
13.2 How sentiment and relevance are produced 24
13.3 Aggregation 25
13.4 How the execution agent uses news, and limitations 26

14 Web application and job runner **26**

15 Verification **28**

1 Purpose and overview

nabia is a platform for researching and running *systematic trend strategies* across asset classes: equity indices, government bond futures, commodities and crypto. A trend strategy of this kind has four stages, and the project provides a component for each:

1. **Data**: daily prices from public sources into a SQL database, kept up to date incrementally (§11).
2. **Signals and views**: filters and forecasters that extract trends from noisy prices (§4, §5). On top of these, statistical analysis and news analysis, which can be done by AI agents (§12).
3. **Allocation**: *risk budgeting* decides each asset's share of portfolio risk. The C++ engine turns budgets into weights, scales them to a volatility target and simulates trading with costs (§7, §8).
4. **Evaluation and operation**: walk-forward backtests without look-ahead, performance metrics, and a web application to run jobs and audit decisions (§9, §14).

The central design decision is to separate *who decides the risk budgets* (a fixed rule, equal risk, or AI agents) from *how budgets become positions* (a deterministic, tested numerical engine). Agents never set weights directly. They choose budgets within bounds, and the engine does the rest.

2 Architecture

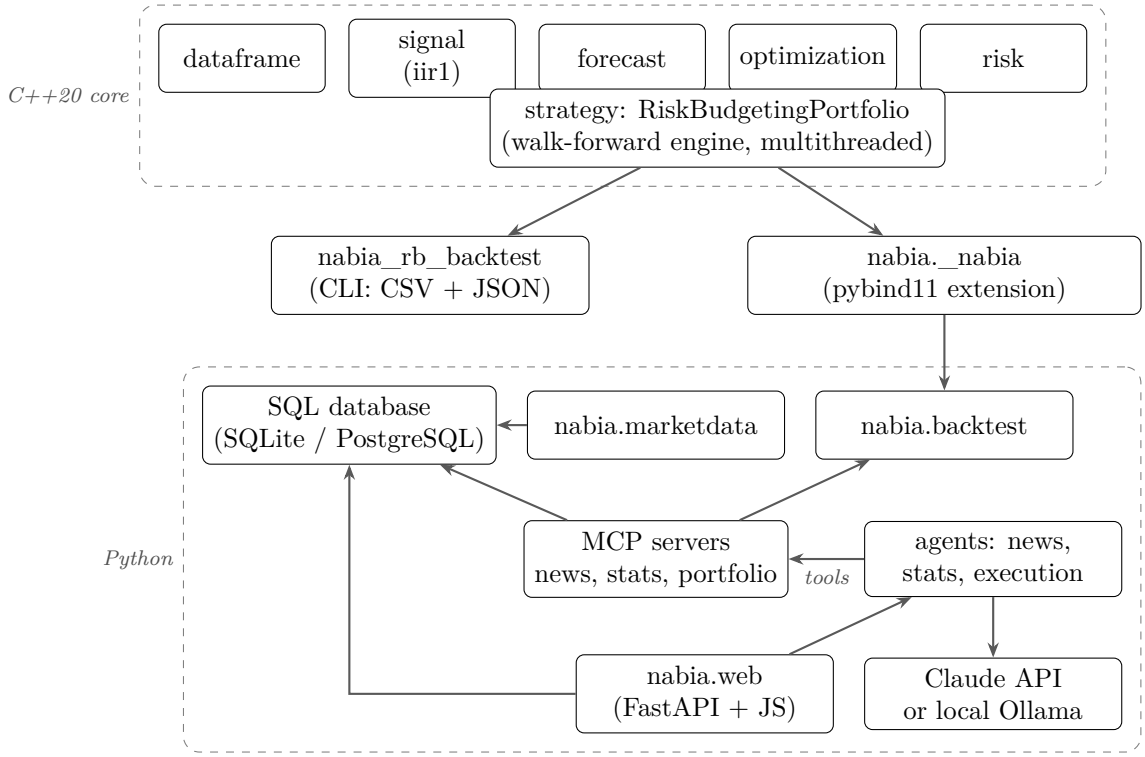


Figure 1: Components and dependencies. Arrows point from a component to what it uses.

2.1 Layers

C++ core (`include/nabia`, `src`) is a static library, `nabia::nabia`, built with CMake (C++20, Eigen for linear algebra, iir1 for filter design). Every module exposes abstract interfaces with a parameter struct owned through `std::unique_ptr`: `IFilter<T>`, `IForecaster<T>`, `IOptimizer<T,R>` and `IRiskMeasure`.

Applications (`apps`): `nabia_rb_backtest` runs a backtest from a prices CSV and a JSON configuration, and writes the results as CSV.

Python bindings (`python/src`): the `nabia._nabia` pybind11 module exposes the engine, and the `nabia` package wraps it in pandas objects.

Python packages `nabia.marketdata` (download and storage), `nabia.ai` (agents, MCP servers, pipeline) and `nabia.web` (web app).

Storage One SQL database holds prices, news, statistics reports, decisions, agent transcripts, jobs and job logs. It goes through SQLAlchemy Core with portable types, so moving from SQLite to PostgreSQL changes only the URL.

2.2 Directory map

Path	Contents
`include/nabia/dataframe`	`TimeSeries`, `DataFrame`, time index concept
`include/nabia/signal`	`IFilter`, `EWMA`, `ButterworthBandPass`, smoothing functions
`include/nabia/forecast`	`IForecaster`, SES, Kalman filter, Gaussian process, kernels
`include/nabia/optimization`	`IOptimizer`; `convex/`: L-BFGS, augmented Lagrangian, SQP; `nongradopt/`: Nelder–Mead, differential evolution
`include/nabia/risk`	`IRiskMeasure`, volatility, VaR, ES, `RiskBudgeting<Measure>`
`include/nabia/strategy`	`RiskBudgetingPortfolio`, performance metrics
`apps/rb_backtest`	command-line backtest (CSV and JSON)
`python/nabia`	Python package: `portfolio`, `marketdata`, `ai`, `web`
`doc`, `docs`	LaTeX sources (built by CMake) and generated PDFs
`tests`, `python/tests`	Catch2 and pytest suites

2.3 Build and dependencies

CMake presets (`debug`, `release`, `asan`, `python`) configure the build with warnings treated as errors. Dependencies are found locally or fetched: Eigen, iir1 and Catch2 for the C++ side, nlohmann/json for the application, pybind11 for the bindings. The LaTeX documents, this one included, are built by a `nabia_docs` target, which first runs `doc/project/extract_snippets.py` to refresh the code excerpts.

3 Time series containers

All numerical code works on Eigen matrices, but financial data is indexed by time. `TimeSeries<Index>` and `DataFrame<Index>` pair an Eigen vector or matrix (rows are times) with a *strictly increasing* index. The index type is any type satisfying the `TimeIndex` concept (copyable, totally ordered): calendar dates (`std::chrono::sys_days`), timestamps, or integers. Construction validates the shapes, the ordering and unique column names. Lookups use binary search.

Two operations matter for the methodology. **Alignment**: markets have different holidays, so a cross-asset backtest keeps only the dates where every asset has a price. **Returns**: simple returns $r_t = P_t/P_{t-1} - 1$ are then computed between consecutive kept dates, so a return can span a gap.

Code 1: Alignment: keep rows where every column is finite
`include/nabia/dataframe/DataFrame.h:108-122`

```
[[nodiscard]] DataFrame drop_missing() const {
    std::vector<Index> index;
    std::vector<Eigen::Index> keep;
    for (Eigen::Index i = 0; i < rows(); ++i) {
        if (values_.row(i).allFinite()) {
            keep.push_back(i);
            index.push_back(index_[static_cast<std::size_t>(i)]);
        }
    }
    Eigen::MatrixXd out(static_cast<Eigen::Index>(keep.size()), cols());
    for (std::size_t k = 0; k < keep.size(); ++k) {
        out.row(static_cast<Eigen::Index>(k)) = values_.row(keep[k]);
    }
    return DataFrame(std::move(index), columns_, std::move(out));
}
```

4 Signal extraction

Filters implement `IFilter<T>`. They are causal (output t uses inputs up to t only) and support both streaming (`process`, one sample at a time) and batch use (`apply`, from a fresh state). Filter design is delegated to the iir1 library, hidden behind a private implementation struct so that iir1 never appears in public headers.

EWMA. The exponentially weighted moving average is the one-pole low-pass $y_t = \alpha x_t + (1-\alpha)y_{t-1}$, i.e. $H(z) = \alpha/(1-(1-\alpha)z^{-1})$. Starting from a zero state biases the first outputs towards zero, so by default the filter is *seeded* with the first sample. Since the DC gain is one, filtering $x_t - x_0$ from a zero state and adding x_0 back is exactly the recursion started at $y_{-1} = x_0$:

$$y_t = x_0 + \alpha \sum_{k \geq 0} (1-\alpha)^k (x_{t-k} - x_0) = \alpha \sum_{k=0}^{t} (1-\alpha)^k x_{t-k} + (1-\alpha)^{t+1} x_0.$$

The helpers `alpha_from_span`$(N) = 2/(N+1)$ (equal mean lag to an N-point moving average) and `alpha_from_halflife`$(h) = 1 - 2^{-1/h}$ convert the usual parameterizations.

Code 2: EWMA on iir1's one-pole section, with first-sample seeding
`src/signal/EWMA.cpp:16-36`

```
struct EWMA::Impl {
    Iir::Custom::OnePole section;
    Iir::DirectFormI state;
    bool seed = true;
    bool started = false;
    double offset = 0.0;

    double process(double x) {
        if (!started) {
            offset = seed ? x : 0.0;
            started = true;
        }
        return section.filter(x - offset, state) + offset;
    }

    void reset() {
        state.reset();
        started = false;
        offset = 0.0;
    }
};
```

Butterworth band-pass. A causal Butterworth band-pass (order N, so $2N$ poles) isolates trend cycles between two frequencies. iir1 is parameterized by a centre and a width, so the edges (f_ℓ, f_h) map to centre $(f_\ell + f_h)/2$ and width $f_h - f_\ell$. Tests verify unit gain inside the band, -3 dB at both edges and strong attenuation outside.

5 Forecasting

Forecasters implement `IForecaster<T>`: `fit(y)`, then `forecast(H)`, which returns a mean and a predictive variance for each step, plus Gaussian prediction intervals. Three models are implemented, each derived with proofs in its own document (`doc/forecast`):

- **Simple exponential smoothing**, with forecast variance $\sigma^2(1 + (h-1)\alpha^2)$ from its ETS(A,N,N) state-space form.
- **Kalman filter** for linear Gaussian state-space models, with ready-made local-level and local-linear-trend models. It uses the Joseph-form covariance update, handles missing data, and provides the likelihood and the Rauch–Tung–Striebel smoother.
- **Gaussian process** regression on time, with squared-exponential, Matérn 5/2, periodic, sum and product kernels. Hyperparameters are fitted by maximizing the marginal likelihood with L-BFGS and analytic gradients.

6 Optimization

Optimizers implement `IOptimizer<T,R>`: `optimize(initial_guess)` returns the solution, and `result()` returns a solver-specific result object.

Convex and smooth. L-BFGS (two-loop recursion with Armijo backtracking) is the workhorse; it is used inside risk budgeting and the Gaussian process. Near a minimum, the decrease that Armijo requires falls below the rounding error in f, and plain backtracking stalls. The line search therefore also accepts Hager–Zhang *approximate Wolfe* steps when f is flat to within rounding: $f(x + td) \le f(x) + \epsilon_f |f(x)|$ and $\sigma g^\top d \le g(x + td)^\top d \le (2c_1 - 1) g^\top d$. On 500-asset risk budgeting problems this took the attainable accuracy from about $3 \cdot 10^{-8}$ to 10^{-8}.

Code 3: L-BFGS line search: Armijo or approximate Wolfe
`src/optimization/convex/LBFGS.cpp:88-116`

```
double t = 1.0;
VectorXd x_new;
VectorXd g_new(x.size());
double f_new = fx;
bool accepted = false;
for (int ls = 0; ls < p.max_line_search; ++ls) {
    x_new = x + t * d;
    f_new = value_and_gradient_(x_new, g_new);
    ++result_.evaluations;
    if (!std::isfinite(f_new)) {
        t *= p.backtrack;
        continue;
    }
    // Armijo sufficient decrease, or Hager-Zhang's approximate Wolfe
    // conditions when f is flat to within rounding: near the minimum
    // the decrease in f drops below machine precision, but the
    // directional derivative is still accurate.
    const double new_slope = g_new.dot(d);
    const bool armijo = f_new <= fx + p.armijo_c1 * t * slope;
    const bool approx_wolfe = f_new <= fx + p.flat_tolerance * std::abs(fx) &&
                              new_slope <= (2.0 * p.armijo_c1 - 1.0) * slope &&
                              new_slope >= p.wolfe_c2 * slope;
    if (armijo || approx_wolfe) {
```

```
        accepted = true;
        break;
    }
    t *= p.backtrack;
}
if (!accepted) {
```

The **augmented Lagrangian** (Powell–Hestenes–Rockafellar) and **SQP** (damped BFGS with an ℓ_1 merit function) solve problems with linear equality, inequality and box constraints. Both are derived in `doc/optimization/convex`.

Derivative-free. **Nelder–Mead** uses adaptive coefficients and enforces box bounds by projecting every trial point. **Differential evolution** (strategies rand/1, best/1, current-to-best/1 and rand/2, binomial crossover, dithered F) evaluates its population in parallel. Trial vectors are generated sequentially from a seeded RNG before evaluation, so results do not depend on the thread count. Evaluation uses a work queue of `std::packaged_task`s consumed by n threads, and the caller collects the results in order through their futures:

Code 4: ParallelEvaluator: job queue, n consumers, results via futures
`src/optimization/nongradopt/parallel_evaluator.hpp:33-76`

```
[[nodiscard]] std::vector<TResult> evaluate(const std::vector<T>& inputs) const {
    std::queue<std::packaged_task<TResult()>> jobs;
    std::vector<std::future<TResult>> futures;
    futures.reserve(inputs.size());
    for (const T& input : inputs) {
        std::packaged_task<TResult()> job([this, &input] { return objective_(input); });
        futures.push_back(job.get_future());
        jobs.push(std::move(job));
    }

    std::mutex mutex;  // guards jobs
    auto consume = [&] {
        for (;;) {
            std::packaged_task<TResult()> job;
            {
                const std::lock_guard lock(mutex);
                if (jobs.empty()) {
                    return;
                }
                job = std::move(jobs.front());
                jobs.pop();
            }
            job();  // stores the result, or the exception, in the job's promise
        }
    };

    // Declared after jobs and mutex: on any exit the consumers are joined
    // (and finish the queue) before what they use is destroyed.
    std::vector<std::jthread> consumers;
    if (num_threads_ == 1) {
        consume();
    } else {
        consumers.reserve(static_cast<std::size_t>(num_threads_));
        for (int t = 0; t < num_threads_; ++t) {
            consumers.emplace_back(consume);
        }
    }
    std::vector<TResult> results;
    results.reserve(inputs.size());
    for (std::future<TResult>& f : futures) {
        results.push_back(f.get());  // waits; rethrows the job's exception
    }
    return results;
}
```

7 Risk measures and risk budgeting

7.1 Risk measures

For a portfolio x (weights) on assets with return covariance Σ and mean μ, with loss $L = -r^\top x$ and $r \sim \mathcal{N}(\mu, \Sigma)$, write $\sigma(x) = \sqrt{x^\top \Sigma x}$. Then:

$$\begin{aligned}
\text{Volatility:} \quad & R(x) = \sigma(x), && \nabla R = \Sigma x/\sigma(x), \\
\text{VaR}_\alpha: \quad & R(x) = -\mu^\top x + z_\alpha\, \sigma(x), && \nabla R = -\mu + z_\alpha \Sigma x/\sigma(x), \\
\text{ES}_\alpha: \quad & R(x) = -\mu^\top x + \frac{\varphi(z_\alpha)}{1-\alpha}\, \sigma(x), && \nabla R = -\mu + \frac{\varphi(z_\alpha)}{1-\alpha} \Sigma x/\sigma(x),
\end{aligned}$$

with $z_\alpha = \Phi^{-1}(\alpha)$. All three are positively homogeneous of degree one, so by Euler's theorem $R(x) = \sum_i x_i\, \partial_i R(x)$: the *risk contributions* $\text{RC}_i = x_i \partial_i R$ add up exactly to the total risk. At $\sigma = 0$ the gradient is set to zero, a valid subgradient of the seminorm σ.

Code 5: Volatility gradient, the building block of every measure
`src/risk/IRiskMeasure.cpp:47-55`

```
Eigen::VectorXd IRiskMeasure::portfolio_volatility_gradient(const Eigen::VectorXd& x) const {
    check_dimension(x, "portfolio");
    Eigen::VectorXd sigma_x = covariance_matrix_ * x;
    const double sigma = std::sqrt(std::max(0.0, x.dot(sigma_x)));
    if (sigma == 0.0) {
        return Eigen::VectorXd::Zero(dim());
    }
    return sigma_x / sigma;
}
```

Code 6: Gaussian Value at Risk and its gradient
`src/risk/measures/ValueAtRisk.cpp:27-32`

```
double ValueAtRisk::compute_risk(const Eigen::VectorXd& x) const {
    return -expected_returns_.dot(x) + z_ * portfolio_volatility(x);
}

Eigen::VectorXd ValueAtRisk::compute_marginal_risk_contributions(const Eigen::VectorXd& x) const {
    return -expected_returns_ + z_ * portfolio_volatility_gradient(x);
```

7.2 Risk budgeting

Given budgets $b_i > 0$ with $\sum b_i = 1$, the risk budgeting portfolio is the long-only x with $\text{RC}_i(x)/R(x) = b_i$. For a convex, homogeneous R that is positive on long-only portfolios, it is obtained from the strictly convex problem

$$\min_{y>0} \; R(y) - \sum_i b_i \log y_i,$$

whose optimality condition $\partial_i R(y) = b_i/y_i$, i.e. $y_i \partial_i R(y) = b_i$, is exactly the budget condition. By Euler's theorem $R(y) = \sum_i b_i = 1$ at the optimum, and $x = y/\sum_i y_i$ normalizes the weights. The log barrier keeps y positive: L-BFGS treats non-positive trial points as infeasible ($+\infty$). Because L-BFGS stops on $\|\nabla f\|$ while the budget error is $\max_i |y_i \nabla_i f|$, the solver warm-restarts with a tighter tolerance until the budget error itself meets the target.

Code 7: Risk budgeting objective and gradient for any risk measure
`src/risk/RiskBudgeting.cpp:65-73`

```
// f(y) = R(y) - b' log(y),  grad f(y) = dR/dy - b ./ y,  y_i grad_i = RC_i(y) - b_i.
// Points with some y_i <= 0 get +inf, which the line search rejects.
const LBFGS::ValueAndGradient objective = [&](const VectorXd& y, VectorXd& grad) {
    if ((y.array() <= 0.0).any()) {
```

```
        grad.setZero();
        return std::numeric_limits<double>::infinity();
    }
    grad = measure.compute_marginal_risk_contributions(y) - b.cwiseQuotient(y);
    return measure.compute_risk(y) - b.dot(VectorXd(y.array().log()));
```

8 The walk-forward portfolio engine

`RiskBudgetingPortfolio` backtests a long-only risk budgeting portfolio from a `DataFrame` of returns (row t holds returns over $(t-1, t]$). Its parameters cover estimation, allocation, execution and accounting.

8.1 Timing: no look-ahead

With covariance window W and rebalance frequency k, rebalances happen at the close of rows $t = W-1, W-1+k, W-1+2k, \ldots$. At rebalance t the engine:

1. estimates moments from rows $t-W+1, \ldots, t$, all known at the close of t;
2. solves for the risk budgeting weights and scales them to the volatility target;
3. trades from the drifted weights to the targets.

The new weights earn the returns of rows $t+1$ onwards. A test confirms there is no look-ahead: changing the data from row 200 on leaves every weight up to row 200 bit-identical.

8.2 Covariance estimation

For the window's returns $r_1, \ldots, r_W$ (oldest first) with weights w_k (equal, or exponential with half-life h: $w_k \propto \lambda^{W-k}$, $\lambda = 2^{-1/h}$, $\sum w_k = 1$):

$$\hat{\mu} = \sum_k w_k r_k, \qquad \hat{\Sigma} = \frac{\sum_k w_k (r_k - \hat{\mu})(r_k - \hat{\mu})^\top}{1 - \sum_k w_k^2},$$

the unbiased weighted estimator. With equal weights it reduces to the usual $1/(W-1)$ sample covariance.

Code 8: Window moments: sample or exponentially weighted
`src/strategy/RiskBudgetingPortfolio.cpp:35-53`

```
WindowMoments estimate(const RiskBudgetingPortfolioParams& p, const MatrixXd& window) {
    const Index W = window.rows();
    VectorXd w = VectorXd::Constant(W, 1.0 / static_cast<double>(W));
    if (p.covariance_estimator == CovarianceEstimator::Exponential) {
        // Weight lambda^age with lambda = 2^(-1 / halflife); the newest row has age 0.
        const double lambda = std::exp2(-1.0 / p.ewma_halflife);
        for (Index i = 0; i < W; ++i) {
            w(i) = std::pow(lambda, static_cast<double>(W - 1 - i));
        }
        w /= w.sum();
    }
    WindowMoments m;
    m.mean = window.transpose() * w;
    const MatrixXd centred = window.rowwise() - m.mean.transpose();
    // Unbiased weighted covariance: divide by 1 - sum(w^2) (= (W - 1) / W for equal weights).
    m.covariance = centred.transpose() * w.asDiagonal() * centred / (1.0 - w.squaredNorm());
    m.covariance = 0.5 * (m.covariance + m.covariance.transpose());
    return m;
}
```

8.3 Allocation and volatility targeting

The budgets in force at rebalance t are the last row of the *budget schedule* dated on or before t; this is how time-varying budgets from a rule or from agents enter the backtest. With no schedule, the parameter budgets apply (equal risk by default). The risk budgeting weights x (sum 1) are then scaled by the leverage

$$L = \min\Big(L_{\max},\ \frac{\sigma^*}{\sqrt{x^\top \hat{\Sigma} x \cdot P}}\Big),$$

where σ^* is the annualized volatility target and P is the number of periods per year. Any weight not invested ($1 - \sum_i w_i$) is cash earning zero.

Code 9: Target weights of one rebalance date
`src/strategy/RiskBudgetingPortfolio.cpp:92-106`

```
Target solve_target(const RiskBudgetingPortfolioParams& p, const VectorXd& budgets, const MatrixXd&
    window,
                    const std::optional<VectorXd>& warm_start) {
    const WindowMoments m = estimate(p, window);
    const RiskBudgetingResult rb = allocate(p, budgets, m, warm_start);
    Target target;
    if (p.target_volatility) {
        const double ppy = static_cast<double>(p.periods_per_year);
        const double predicted = std::sqrt(std::max(0.0, rb.weights.dot(m.covariance * rb.weights)) *
    ppy);
        target.leverage = predicted > 0.0 ? std::min(p.max_leverage, *p.target_volatility / predicted)
    : p.max_leverage;
    }
    target.weights = target.leverage * rb.weights;
    target.contributions = rb.relative_contributions;
    target.converged = rb.converged;
    return target;
}
```

8.4 Parallel solves, sequential accounting

The targets of all rebalance dates depend only on their own windows, so phase 1 solves them in parallel. To keep warm starts (each solve starts from the previous date's weights) without making results depend on the thread count, dates are grouped into fixed blocks of 8 consecutive rebalances: one block is one task, and warm starts happen only within a block. Results are therefore bit-identical for any number of threads. On 50 assets with daily rebalancing over 10 years (2,271 solves), the backtest takes 375 ms on one thread and 53 ms on eight.

Code 10: Phase 1: target weights of every rebalance date, in parallel
`src/strategy/RiskBudgetingPortfolio.cpp:192-208`

```
// Phase 1 (parallel): target weights of every rebalance date. Each task
// solves one block of consecutive dates, warm-starting within the block.
std::vector<Target> targets(static_cast<std::size_t>(n_rebalances));
const Index n_blocks = (n_rebalances + kWarmStartBlock - 1) / kWarmStartBlock;
const int threads = nabia::detail::resolve_threads(p.num_threads, n_blocks);
nabia::detail::parallel_for(n_blocks, threads, [&](std::int64_t block) {
    std::optional<VectorXd> warm_start;
    const Index first = block * kWarmStartBlock;
    const Index last = std::min(n_rebalances, first + kWarmStartBlock);
    for (Index k = first; k < last; ++k) {
        const Index t = c.rebalance_rows[static_cast<std::size_t>(k)];
        Target target = solve_target(p, budgets_at(t), returns.middleRows(t - W + 1, W), warm_start);
        warm_start = target.weights / target.leverage;  // the unlevered risk budgeting weights
        targets[static_cast<std::size_t>(k)] = std::move(target);
    }
});
c.threads_used = threads;
```

Code 11: The thread pool helper used by phase 1
`src/common/parallel_for.hpp:27-57`

```cpp
void parallel_for(std::int64_t count, int threads, const Task& task) {
    std::atomic<std::int64_t> next{0};
    std::mutex error_mutex;
    std::exception_ptr error;
    std::int64_t error_index = count;

    auto work = [&] {
        for (std::int64_t i = next++; i < count; i = next++) {
            try {
                task(i);
            } catch (...) {
                const std::lock_guard lock(error_mutex);
                if (i < error_index) {
                    error_index = i;
                    error = std::current_exception();
                }
            }
        }
    };
    {
        std::vector<std::jthread> workers;
        workers.reserve(static_cast<std::size_t>(std::max(0, threads - 1)));
        for (int t = 1; t < threads; ++t) {
            workers.emplace_back(work);
        }
        work();
    }  // joining the workers makes all their writes visible here
    if (error) {
        std::rethrow_exception(error);
    }
}
```

8.5 Drift, trading costs and accounting

Phase 2 walks through time. With held weights w_t and asset returns r_t:

$$
\begin{aligned}
g_t &= w_t^\top r_t && \text{gross return,}\\
w &\leftarrow w_t \odot (1 + r_t)/(1 + g_t) && \text{drift (holdings grow, cash does not),}\\
\tau_t &= \|w^{\text{target}} - w\|_1, \quad c_t = \kappa\,\tau_t && \text{turnover and proportional cost,}\\
n_t &= (1 + g_t)(1 - c_t) - 1 && \text{net return; equity } E_t = E_{t-1}(1 + n_t).
\end{aligned}
$$

Costs are paid out of equity at the close. The first trade buys the whole portfolio from cash, so its turnover equals the sum of the weights. These identities are checked to 10^{-15} in the tests.

Code 12: Phase 2: earn returns, drift, trade, account
`src/strategy/RiskBudgetingPortfolio.cpp:210-254`

```cpp
// Phase 2 (sequential): drift, trade and account through time.
VectorXd w = VectorXd::Zero(N);  // current weights (fractions of equity)
double equity = p.initial_capital;
const double ppy = static_cast<double>(p.periods_per_year);
Index next = 0;  // next rebalance

for (Index t = 0; t < T; ++t) {
    // 1. Earn this period's returns with the weights set at the previous close.
    c.weights.row(t) = w.transpose();
    const VectorXd r = returns.row(t).transpose();
    const double gross = w.dot(r);
    if (!(1.0 + gross > 0.0)) {
        throw std::invalid_argument("RiskBudgetingPortfolio: the portfolio lost all its capital at row
     " +
                                    std::to_string(t));
    }
    if (p.drift_weights) {
        // Each holding grows by (1 + r_i), equity by (1 + gross); cash is the remainder.
```

```
        w = w.cwiseProduct((VectorXd::Ones(N) + r)) / (1.0 + gross);
    }

    // 2. Rebalance at the close, with the data known at t.
    double cost = 0.0;
    if (next < n_rebalances && c.rebalance_rows[static_cast<std::size_t>(next)] == t) {
        const Target& target = targets[static_cast<std::size_t>(next)];
        const double traded = (target.weights - w).lpNorm<1>();
        cost = p.transaction_cost * traded;
        w = target.weights;

        c.target_weights.row(next) = target.weights.transpose();
        c.risk_contributions.row(next) = target.contributions.transpose();
        c.leverage(next) = target.leverage;
        c.all_converged = c.all_converged && target.converged;
        c.turnover(t) = traded;
        ++next;
    }

    // 3. Accounting: costs are paid out of the equity at the close.
    const double net = (1.0 + gross) * (1.0 - cost) - 1.0;
    const double new_equity = equity * (1.0 + net);
    c.gross_returns(t) = gross;
    c.costs(t) = cost;
    c.net_returns(t) = net;
    c.pnl(t) = new_equity - equity;
    c.equity(t) = new_equity;
    equity = new_equity;
```

9 Performance metrics

From the net returns $n_1, \dots, n_T$ (from the first trade on) and P periods per year:

Metric	Definition
Total return	$\prod_t (1 + n_t) - 1$
Annualized return (CAGR)	$(1 + \text{total})^{P/T} - 1$
Annualized volatility	$\text{sd}(n)\sqrt{P}$ (sample standard deviation)
Sharpe ratio	$\bar{n}\,P/(\text{sd}(n)\sqrt{P})$ (zero risk-free rate)
Sortino ratio	$\bar{n}\,P/\big(\sqrt{\frac{1}{T}\sum_t \min(n_t, 0)^2}\sqrt{P}\big)$
Maximum drawdown	$\max_t \left(1 - E_t / \max_{s \le t} E_s\right)$
Max drawdown duration	longest run of periods below a previous peak
Calmar ratio	annualized return / maximum drawdown
Hit rate	fraction of periods with $n_t > 0$
Turnover	average τ per rebalance; annualized $\sum_t \tau_t\, P/T$

Code 13: Performance metrics
`src/strategy/PerformanceMetrics.cpp:31-67`

```
PerformanceMetrics compute_performance(const Eigen::VectorXd& returns, int periods_per_year) {
    if (returns.size() == 0 || !returns.allFinite()) {
        throw std::invalid_argument("compute_performance: returns must be non-empty and finite");
    }
    if (periods_per_year <= 0) {
        throw std::invalid_argument("compute_performance: periods_per_year must be positive");
    }
    const auto n = static_cast<double>(returns.size());
    const double ppy = static_cast<double>(periods_per_year);
    PerformanceMetrics m;
    m.periods = returns.size();

    const Eigen::VectorXd equity = equity_curve(returns, 1.0);
    m.total_return = equity(equity.size() - 1) - 1.0;
    m.annualized_return = m.total_return > -1.0 ? std::pow(1.0 + m.total_return, ppy / n) - 1.0 : -1.0;
```

```
    const double mean = returns.mean();
    const double var = returns.size() > 1 ? (returns.array() - mean).square().sum() / (n - 1.0) : 0.0;
    m.annualized_volatility = std::sqrt(var * ppy);
    m.sharpe_ratio = m.annualized_volatility > 0.0 ? mean * ppy / m.annualized_volatility : 0.0;
    const double downside = std::sqrt(returns.cwiseMin(0.0).squaredNorm() / n) * std::sqrt(ppy);
    m.sortino_ratio = downside > 0.0 ? mean * ppy / downside : 0.0;

    const Eigen::VectorXd dd = drawdown_series(equity, 1.0);
    m.max_drawdown = dd.maxCoeff();
    Eigen::Index run = 0;
    for (Eigen::Index t = 0; t < dd.size(); ++t) {
        run = dd(t) > 0.0 ? run + 1 : 0;
        m.max_drawdown_duration = std::max(m.max_drawdown_duration, run);
    }
    m.calmar_ratio = m.max_drawdown > 0.0 ? m.annualized_return / m.max_drawdown : 0.0;

    m.hit_rate = static_cast<double>((returns.array() > 0.0).count()) / n;
    m.best_period = returns.maxCoeff();
    m.worst_period = returns.minCoeff();
    return m;
}
```

10 Command-line application and Python bindings

`nabia_rb_backtest` reads prices from a CSV (ISO dates, one column per asset; missing values are dropped by date alignment) and portfolio parameters from JSON. It writes a CSV row per date with the weights, risk contributions on rebalance dates, turnover, costs, returns, PnL, equity and drawdown. Unknown JSON keys are rejected, so typos fail loudly.

The Python module reuses exactly this pipeline and the same JSON parser, so Python and the CLI accept the same parameters with the same error messages. Dates cross the boundary as `int64` days since 1970-01-01 (numpy `datetime64[D]`), prices as a `float64` matrix, and parameters as JSON text. The GIL is released during the C++ computation, so the engine's threads run in parallel.

Code 14: Shared preparation: parse, align, window, returns, budgets
`python/src/nabia_module.cpp:65-96`

```
Prepared prepare(const DaysArray& days, const std::vector<std::string>& assets, const MatrixXd& prices,
                 const std::string& config_json) {
    if (prices.rows() != days.size() || prices.cols() != static_cast<Index>(assets.size())) {
        throw std::invalid_argument("prices must be (dates x assets): got " + std::to_string(prices.
    rows()) + " x " +
                                    std::to_string(prices.cols()) + " for " + std::to_string(days.size
    ()) +
                                    " dates and " + std::to_string(assets.size()) + " assets");
    }
    Prepared p;
    p.config = nabia::app::parse_config(config_json);
    DataFrame<Date> px(to_dates(days), assets, prices);
    const Index read = px.rows();
    px = px.drop_missing();
    p.dropped_dates = read - px.rows();
    if (px.rows() >= 2 && (p.config.start || p.config.end)) {
        px = px.between(p.config.start.value_or(px.time(0)), p.config.end.value_or(px.time(px.rows() -
    1)));
    }
    if (px.rows() < 2) {
        throw std::invalid_argument("need at least 2 dates with a price for every asset");
    }
    for (Index t = 0; t < px.rows(); ++t) {
        for (Index j = 0; j < px.cols(); ++j) {
            if (!(px.values()(t, j) > 0.0)) {
                throw std::invalid_argument("prices must be positive for returns: " + px.columns()[
    static_cast<std::size_t>(j)] +
                                            " is " + std::to_string(px.values()(t, j)) + " on " +
                                            nabia::app::format_date(px.time(t)));
            }
```

```
        }
    }
    p.returns = px.pct_change();
    p.config.params.budgets = nabia::app::resolve_budgets(p.config, assets);
    return p;
}
```

Code 15: The engine call, with the GIL released
`python/src/nabia_module.cpp:133-138`

```
std::unique_ptr<PortfolioResult<Date>> r;
{
    // Pure C++ from here on: let other Python threads run meanwhile.
    const py::gil_scoped_release release;
    const RiskBudgetingPortfolio engine(std::make_unique<RiskBudgetingPortfolioParams>(p.config.params)
    );
    r = std::make_unique<PortfolioResult<Date>>(schedule ? engine.run(p.returns, *schedule) : engine.
    run(p.returns));
```

Code 16: pandas to C++: dates, names, values, JSON parameters
`python/nabia/portfolio.py:104-110`

```
def _inputs(prices: pd.DataFrame, params: Mapping[str, Any] | None) -> tuple[np.ndarray, list[str], np.
    ndarray, str]:
    if not isinstance(prices, pd.DataFrame):
        raise TypeError("prices must be a pandas DataFrame (dates x assets)")
    if prices.shape[1] == 0:
        raise ValueError("prices has no columns")
    values = np.ascontiguousarray(prices.to_numpy(dtype=np.float64, na_value=np.nan))
    return _to_days(prices.index), [str(c) for c in prices.columns], values, _config_json(params)
```

11 Market data

11.1 Universe and sources

The default universe has four asset classes: equity indices (S&P 500, Nasdaq 100, Euro Stoxx 50, DAX, FTSE, Nikkei, Hang Seng and others), US Treasury futures (2Y to Ultra bond), commodity futures (energy, metals, grains) and spot crypto (BTC, ETH, SOL, XRP, ADA, LTC against USD). Each asset records which source provides it:

- **Yahoo Finance** (through `yfinance`) for indices and futures. Futures tickers are front-month continuous series that jump at contract rolls. WTI went negative on 2020-04-20, where simple returns are undefined, so the examples use Brent.
- **Crypto exchanges** through `ccxt` (Coinbase by default; public data, no key). History is paged in windows of 300 candles. Windows before a coin's listing date come back empty and are stepped over, and paging stops if the exchange ignores the start date.

Code 17: Crypto daily candles with pagination
`python/nabia/marketdata/sources.py:147-188`

```
def fetch(self, symbols: Sequence[str], start: dt.date, end: dt.date | None = None) -> pd.DataFrame:
    client = self._exchange()
    start_ms = int(dt.datetime.combine(start, dt.time(), dt.UTC).timestamp() * 1000)
    last_day = end or dt.datetime.now(dt.UTC).date()
    stop_ms = int(dt.datetime.combine(last_day + dt.timedelta(days=1), dt.time(), dt.UTC).timestamp() *
      1000)
    frames = []
    for symbol in symbols:
        if symbol not in client.markets:
            log.warning("%s is not listed on %s", symbol, getattr(client, "id", "the exchange"))
            continue
        candles: dict[int, list[float]] = {}
```

```
        since = start_ms
        while since < stop_ms:
            page = self._page(client, symbol, since)
            if not page:
                # Nothing in this window: before the listing date (or a gap). Step
                # forward a page; stop once past what has been collected so far.
                if candles:
                    break
                since += self.page_size * _DAY_MS
                continue
            for candle in page:
                if start_ms <= candle[0] < stop_ms:
                    candles[int(candle[0])] = candle
            next_since = int(page[-1][0]) + _DAY_MS
            if next_since <= since:  # no progress: the exchange ignores 'since'
                break
            since = next_since
        if not candles:
            continue
        rows = [candles[k] for k in sorted(candles)]
        frames.append(pd.DataFrame({
            "symbol": symbol,
            "date": [dt.datetime.fromtimestamp(r[0] / 1000, dt.UTC).date() for r in rows],
            "open": [r[1] for r in rows],
            "high": [r[2] for r in rows],
            "low": [r[3] for r in rows],
            "close": [r[4] for r in rows],
            "adj_close": [r[4] for r in rows],
            "volume": [r[5] for r in rows],
        }).astype({c: "float64" for c in PRICE_COLUMNS[2:]}))
    return pd.concat(frames, ignore_index=True)[PRICE_COLUMNS] if frames else empty_prices()
```

11.2 Storage and incremental updates

Prices are stored per (symbol, date) with open, high, low, close, adjusted close and volume. Writes are *upserts* (`INSERT ... ON CONFLICT DO UPDATE`, in SQLite and PostgreSQL syntax), so re-downloading is idempotent and picks up revisions. Updates are incremental: a stored symbol is re-fetched from five days before its last date, so recent revisions are corrected. Symbols are grouped by source and start date to batch requests.

Code 18: Portable upsert (SQLite and PostgreSQL)
`python/nabia/marketdata/store.py:66-81`

```
def upsert_rows(engine: Engine, table: Table, rows: list[dict], keys: Sequence[str], chunk: int = 500)
     -> None:
    """Insert rows, updating the non-key columns of rows whose keys already exist."""
    if not rows:
        return
    dialect = engine.dialect.name
    if dialect == "sqlite":
        from sqlalchemy.dialects.sqlite import insert
    elif dialect == "postgresql":
        from sqlalchemy.dialects.postgresql import insert
    else:  # pragma: no cover - other databases are not supported yet
        raise NotImplementedError(f"upsert is not implemented for {dialect}")
    with engine.begin() as connection:
        for i in range(0, len(rows), chunk):
            statement = insert(table).values(rows[i : i + chunk])
            updates = {c.name: statement.excluded[c.name] for c in table.columns if c.name not in keys}
            connection.execute(statement.on_conflict_do_update(index_elements=list(keys), set_=updates)
     )
```

Code 19: Incremental download with overlap, routed by source
`python/nabia/marketdata/download.py:19-72`

```
def update_market_data(
    store: MarketDataStore,
    source: MarketDataSource | Mapping[str, MarketDataSource],
```

```
    assets: Sequence[Asset],
    start: dt.date,
    end: dt.date | None = None,
    full: bool = False,
    overlap_days: int = DEFAULT_OVERLAP_DAYS,
) -> dict[str, int]:
    """Downloads daily prices for ``assets`` and upserts them into ``store``.

    ``source`` is either one source for every asset, or a mapping from source
    name to source, in which case each asset is fetched from ``asset.source``
    (e.g. ``{"yahoo": YahooFinanceSource(), "crypto": CryptoSource()}``).

    Incremental by default: a symbol already in the store is fetched from
    ``overlap_days`` before its last stored date (so recent revisions are
    picked up); a new symbol from ``start``. ``full=True`` re-downloads
    everything from ``start``. Returns the number of rows written per symbol
    (0 for symbols the source returned no data for).
    """
    sources: dict[str, MarketDataSource]
    if isinstance(source, Mapping):
        sources = dict(source)
        unknown = {a.source for a in assets} - sources.keys()
        if unknown:
            raise ValueError(f"no source given for: {sorted(unknown)}")
        source_of = {a.symbol: a.source for a in assets}
    else:
        sources = {source.name: source}
        source_of = {a.symbol: source.name for a in assets}
    for name in sorted({source_of[a.symbol] for a in assets}):
        store.upsert_assets([a for a in assets if source_of[a.symbol] == name], source=name)
    last = {} if full else store.last_dates()

    # Group symbols by source and fetch start, so each group is one batched request.
    groups: dict[tuple[str, dt.date], list[str]] = {}
    for asset in assets:
        from_date = start
        if asset.symbol in last:
            from_date = max(start, last[asset.symbol] - dt.timedelta(days=overlap_days))
        groups.setdefault((source_of[asset.symbol], from_date), []).append(asset.symbol)

    written = {asset.symbol: 0 for asset in assets}
    for (name, from_date), symbols in sorted(groups.items()):
        log.info("fetching %d symbols from %s since %s", len(symbols), name, from_date)
        prices = sources[name].fetch(symbols, from_date, end)
        for symbol, rows in prices.groupby("symbol"):
            written[str(symbol)] = store.upsert_prices(rows)
            log.info("%s: %d prices stored (%s to %s)", symbol, written[str(symbol)], rows["date"].min
     (), rows["date"].max())
    for symbol, count in written.items():
        if count == 0:
            log.warning("no data for %s", symbol)
    return written
```

12 AI agents

12.1 Roles and data flow

Three agents share one database. Each agent reaches the database only through tools served by local MCP servers:

Agent	Tools	Output
News analyzer	`news` server (RSS feeds, store, query, summary); web search and fetch on Claude	articles with topic, assets, sentiment, relevance (§13)
Stats analyzer	`stats` server (correlations, volatilities, trend signals, regime, reports)	a structured report with a budget tilt per asset
Execution agent	`news` and `stats` (read-only), `portfolio` server	risk budgets with a rationale, and target weights from the C++ engine

The daily pipeline runs news, then stats, then execution. In backtest mode only stats and execution run, at each decision date (§12.7).

12.2 MCP servers and point-in-time tools

Each server is a standalone stdio MCP server (`python -m nabia.ai.servers.<name>`), usable by the agents or by any MCP client. Three properties matter:

- **As-of discipline.** Every analytical tool takes an `as_of` date and reads only data dated on or before it. A test rewrites all prices after `as_of` and checks that every tool's output is unchanged.
- **Errors the model can use.** Anticipated failures (wrong symbols, out-of-range values) are raised as MCP `ToolError`s with their message, so the model sees why a call failed and can correct it. Any other exception would reach the model only as a generic error.
- **Read-only mode** removes writing tools. The execution agent cannot store news or overwrite statistics reports.

Code 20: The as-of date bounding every query
`python/nabia/ai/servers/common.py:52-60`

```python
def parse_as_of(text: str) -> dt.date:
    """'YYYY-MM-DD' (or 'today'). The as-of date bounds every query: tools never
    return data dated after it, which keeps backtests free of look-ahead."""
    if text == "today":
        return dt.date.today()
    try:
        return dt.date.fromisoformat(text)
    except ValueError as error:
        raise ValueError(f"as_of must be YYYY-MM-DD, got {text!r}") from error
```

Code 21: Tool registration: anticipated errors reach the model
`python/nabia/ai/servers/common.py:94-114`

```python
def tool(server: MCPServer) -> Callable[[F], F]:
    """Registers a function as an MCP tool, reporting ValueError / KeyError to the
    model as tool errors with their message (MCP otherwise treats them as crashes
    and hides the text), so the agent can correct its call."""

    def decorator(fn: F) -> F:
        @functools.wraps(fn)
        def wrapper(*args: Any, **kwargs: Any) -> Any:
            try:
                return fn(*args, **kwargs)
            except (ValueError, KeyError) as error:
                raise ToolError(str(error)) from error

        # Resolved annotations, so schema generation does not depend on this module's globals.
        wrapper.__signature__ = inspect.signature(fn, eval_str=True)  # type: ignore[attr-defined]
        wrapper.__annotations__ = {**{k: p.annotation for k, p in wrapper.__signature__.parameters.
    items()},
```

```
                                "return": wrapper.__signature__.return_annotation}
        server.tool()(wrapper)
        return fn

    return decorator
```

12.3 The agent loop

The runtime starts the agent's MCP servers as subprocesses, lists their tools, and runs a manual tool-use loop. It handles parallel tool calls (all results go back in one message), paused turns of hosted tools, refusals and turn limits, and records every call for the audit trail.

Claude (`provider: "anthropic"`): `claude-opus-5-5` with adaptive thinking, a per-agent effort level, prompt caching of the stable prefix, and server-side refusal fallbacks.

Local (`provider: "ollama"`): an adapter with the same interface translates the requests and replies to Ollama's chat API: tool definitions, tool calls as `tool_calls`, tool results as `tool` messages. Claude-only options are ignored, and hosted web tools are dropped (local news comes from RSS).

Code 22: The agent loop
`python/nabia/ai/agents/runtime.py:150-195`

```
async def _loop(self, record: AgentRun, system: str, prompt: str, tools: list[dict[str, Any]],
                sessions: dict[str, ClientSession], effort: str) -> None:
    messages: list[dict[str, Any]] = [{"role": "user", "content": prompt}]
    for turn in range(1, self.max_turns + 1):
        log.info("%s: turn %d, waiting for the model...", record.agent, turn)
        started = time.monotonic()
        response = await self.client.beta.messages.create(
            model=self.config.model,
            max_tokens=MAX_TOKENS,
            system=system,
            tools=tools,
            messages=messages,
            thinking={"type": "adaptive"},
            output_config={"effort": effort},
            cache_control={"type": "ephemeral"},  # caches the stable prefix: tools, system, history
            betas=["server-side-fallback-2026-07-01"],
            fallbacks="default",  # re-run a refused request on a fallback model
        )
        record.turns += 1
        self._add_usage(record, response.usage)
        log.info("%s: turn %d -> %s (%s in / %s out tokens, %.1fs)", record.agent, turn, response.
    stop_reason,
                 getattr(response.usage, "input_tokens", "?"), getattr(response.usage, "output_tokens",
     "?"),
                 time.monotonic() - started)
        for block in response.content:
            if block.type == "text" and block.text.strip() and response.stop_reason == "tool_use":
                log.info("%s says: %s", record.agent, _preview(block.text, 300))
            elif block.type == "server_tool_use":
                log.info("%s -> %s %s", record.agent, block.name, _preview(getattr(block, "input", {}))
    )
        if response.stop_reason == "refusal":
            details = getattr(response, "stop_details", None)
            raise AgentRefusedError(f"{record.agent} refused: {getattr(details, 'category', None)}")
        messages.append({"role": "assistant", "content": response.content})
        record.server_tool_calls += sum(1 for b in response.content if b.type == "server_tool_use")
        if response.stop_reason == "pause_turn":  # a long server-tool turn: resend to resume it
            continue
        if response.stop_reason == "max_tokens":
            raise RuntimeError(f"{record.agent}: response hit max_tokens ({MAX_TOKENS})")
        if response.stop_reason != "tool_use":
            record.final_text = "\n".join(b.text for b in response.content if b.type == "text").strip()
            log.info("%s finished after %d turns, %d tool calls: %s", record.agent, record.turns,
                     len(record.tool_calls), _preview(record.final_text, 400))
```

```
            return
        calls = [b for b in response.content if b.type == "tool_use"]
        results = await asyncio.gather(*(self._call_tool(record, sessions, b) for b in calls))
        messages.append({"role": "user", "content": list(results)})  # all results in one message
    raise RuntimeError(f"{record.agent}: no final answer after {self.max_turns} turns")
```

Code 23: Executing one tool call on its MCP server
python/nabia/ai/agents/runtime.py:197-215

```
async def _call_tool(self, record: AgentRun, sessions: dict[str, ClientSession], block: Any) -> dict[
    str, Any]:
    session = sessions.get(block.name)
    log.info("%s -> %s(%s)", record.agent, block.name, _preview(dict(block.input)))
    try:
        if session is None:
            raise ValueError(f"unknown tool {block.name}")
        result = await session.call_tool(block.name, dict(block.input))
        text = "\n".join(c.text for c in result.content if getattr(c, "type", "") == "text")
        if not text and getattr(result, "structured_content", None) is not None:
            text = json.dumps(result.structured_content)
        is_error = bool(result.is_error)
    except Exception as error:  # a broken server must not kill the agent: report it as a tool error
        text, is_error = f"{type(error).__name__}: {error}", True
    record.tool_calls.append(ToolCall(block.name, dict(block.input), is_error, text[:2000]))
    if is_error:
        log.warning("%s <- %s failed: %s", record.agent, block.name, _preview(text, 300))
    else:
        log.info("%s <- %s: %s", record.agent, block.name, _preview(text))
    return {"type": "tool_result", "tool_use_id": block.id, "content": text or "(no output)", "is_error
    ": is_error}
```

Code 24: Ollama replies translated to Anthropic-style blocks
python/nabia/ai/agents/ollama.py:120-146

```
def _response(self, data: dict[str, Any]) -> SimpleNamespace:
    message = data.get("message", {})
    content: list[SimpleNamespace] = []
    text = (message.get("content") or "").strip()
    if text:
        content.append(SimpleNamespace(type="text", text=text))
    for call in message.get("tool_calls") or []:
        function = call.get("function", {})
        arguments = function.get("arguments") or {}
        if isinstance(arguments, str):  # some models return a JSON string
            try:
                arguments = json.loads(arguments)
            except json.JSONDecodeError:
                arguments = {"_unparsed": arguments}
        call_id = call.get("id") or f"call_{next(self._ids)}"
        self._tool_names[call_id] = function.get("name", "")
        content.append(SimpleNamespace(type="tool_use", id=call_id, name=function.get("name", ""),
    input=arguments))
    if any(b.type == "tool_use" for b in content):
        stop_reason = "tool_use"
    elif data.get("done_reason") == "length":
        stop_reason = "max_tokens"
    else:
        stop_reason = "end_turn"
    usage = SimpleNamespace(input_tokens=data.get("prompt_eval_count", 0), output_tokens=data.get("
    eval_count", 0),
                            cache_read_input_tokens=0, cache_creation_input_tokens=0)
    return SimpleNamespace(content=content, stop_reason=stop_reason, usage=usage, stop_details=None,
                           model=data.get("model", self.model))
```

12.4 Statistics computed for the stats analyzer

All statistics come from daily close prices up to `as_of`. Volatilities are annualized with $\sqrt{252}$.

- **Aligned returns** for cross-asset statistics: dates where all assets have a price, then simple

returns.

- **Correlation** of the last w aligned returns (60 by default) and the **average correlation** $\frac{1}{n(n-1)} \sum_{i \neq j} \rho_{ij}$.
- **Rolling volatility** of each asset's own returns over 20, 60 and 120 days, and the **volatility percentile**: the fraction of the last 252 rolling 20-day volatilities that are at or below today's.
- **Trend signals**: returns over 21, 63, 126 and 252 observations; distance from the 50- and 200-day moving averages, $P_T / \overline{P}_n - 1$; the **volatility-scaled 12-1 momentum**

$$m = \frac{P_{T-21}/P_{T-252} - 1}{\mathrm{sd}(r_{T-251..T})\sqrt{252}},$$

  which skips the last month (short-term reversal) and measures the trend in units of the asset's own risk; and the current and 1-year maximum drawdowns.
- **Market regime**: the average correlation over 60 and 252 days, the median volatility percentile, and the share of assets above their 200-day average or up over 12 months.

Code 25: Trend signals
python/nabia/ai/analytics.py:51-75

```python
def trend_signals(prices: pd.DataFrame) -> pd.DataFrame:
    """Per symbol: past returns (1m, 3m, 6m, 12m), distance from the 50- and
    200-day moving averages, a volatility-scaled 12-1 momentum score, and the
    current and 1-year maximum drawdowns."""
    out = {}
    for symbol in prices.columns:
        p = prices[symbol].dropna()
        last = p.iloc[-1]
        row: dict[str, float] = {}
        for label, n in (("ret_1m", 21), ("ret_3m", 63), ("ret_6m", 126), ("ret_12m", 252)):
            row[label] = float(last / p.iloc[-n - 1] - 1) if len(p) > n else float("nan")
        for n in (50, 200):
            row[f"vs_ma{n}"] = float(last / p.tail(n).mean() - 1) if len(p) >= n else float("nan")
        r = p.pct_change().dropna()
        vol = r.tail(252).std() * np.sqrt(PERIODS_PER_YEAR)
        if len(p) > 252 and vol > 0:
            row["momentum_12_1"] = float((p.iloc[-22] / p.iloc[-253] - 1) / vol)  # skip the last month
        else:
            row["momentum_12_1"] = float("nan")
        year = p.tail(253)
        drawdown = 1 - year / year.cummax()
        row["drawdown"] = float(drawdown.iloc[-1])
        row["max_drawdown_1y"] = float(drawdown.max())
        out[symbol] = row
    return pd.DataFrame(out).T
```

Code 26: Volatilities and their percentile
python/nabia/ai/analytics.py:36-48

```python
def rolling_volatility(prices: pd.DataFrame, windows: tuple[int, ...] = (20, 60, 120)) -> pd.DataFrame:
    """Per symbol: annualized volatility over each window (its own returns, not
    aligned with the other assets), plus where the 20-day volatility sits in
    its last year of history (percentile, 0..1)."""
    out = {}
    for symbol in prices.columns:
        r = prices[symbol].dropna().pct_change().dropna()
        row = {f"vol_{w}d": float(r.tail(w).std() * np.sqrt(PERIODS_PER_YEAR)) if len(r) >= w else
     float("nan")
               for w in windows}
        rolling = r.rolling(20).std().dropna().tail(PERIODS_PER_YEAR)
        row["vol_20d_percentile_1y"] = float((rolling <= rolling.iloc[-1]).mean()) if len(rolling) else
      float("nan")
```

```
        out[symbol] = row
    return pd.DataFrame(out).T
```

12.5 From decisions to positions

The execution agent proposes positive relative budgets with a written rationale. The tool `set_risk_budgets` normalizes them and enforces the bounds $[b_{\min}, b_{\max}]$ by iterative projection: values beyond a bound are fixed at it, and the rest are rescaled to fill the remainder, until nothing is out of bounds. `compute_target_weights` then runs the C++ engine on prices up to `as_of` (`nabia.latest_weights`) and stores the weights with the decision. Each decision records the model that made it, e.g. `agent:ollama:qwen3.8:latest`.

Code 27: Budget normalization within bounds
python/nabia/ai/analytics.py:107-139

```
def project_budgets(budgets: dict[str, float], min_budget: float, max_budget: float) -> dict[str, float
    ]:
    """Normalizes positive budgets to sum to 1 with every value in [min_budget, max_budget].

    Values beyond a bound are fixed at that bound and the rest rescaled to fill
    the remainder, repeating until nothing is out of bounds.
    """
    if not budgets:
        raise ValueError("no budgets")
    if any(not np.isfinite(v) or v <= 0 for v in budgets.values()):
        raise ValueError("budgets must be positive numbers")
    n = len(budgets)
    if not (min_budget * n <= 1.0 + 1e-12 and max_budget * n >= 1.0 - 1e-12):
        raise ValueError("bounds are infeasible for this number of assets")
    fixed: dict[str, float] = {}
    free = dict(budgets)
    while True:
        remaining = 1.0 - sum(fixed.values())
        total = sum(free.values())
        scaled = {k: v * remaining / total for k, v in free.items()}
        low = {k for k, v in scaled.items() if v < min_budget - 1e-12}
        high = {k for k, v in scaled.items() if v > max_budget + 1e-12}
        if not low and not high:
            result = {**fixed, **scaled}
            return {k: result[k] for k in budgets}
        for k in low:
            fixed[k] = min_budget
            free.pop(k)
        for k in high:
            fixed[k] = max_budget
            free.pop(k)
        if not free:
            result = {k: fixed[k] / sum(fixed.values()) for k in budgets}
            return result
```

Code 28: The decision tool
python/nabia/ai/servers/portfolio.py:45-58

```
@tool(server)
def set_risk_budgets(as_of: str, budgets: dict[str, float], rationale: str) -> dict:
    """Decide the risk budgets for as_of: one positive number per portfolio symbol
    (relative sizes; normalized to sum to 1 and bounded to [min_budget, max_budget]).
    The rationale should cite the evidence (stats, news) behind each tilt."""
    end = parse_as_of(as_of)
    _check_symbols(ctx, budgets)
    if len(rationale.strip()) < 20:
        raise ValueError("give a rationale that explains the budgets")
    projected = analytics.project_budgets(budgets, ctx.config.min_budget, ctx.config.max_budget)
    ctx.store.save_decision(ctx.run, end, projected, rationale.strip(), decided_by=f"agent:{ctx.config.
     model_label}")
    changed = {s: round(projected[s] - budgets[s] / sum(budgets.values()), 4) for s in projected}
    return clean({"as_of": end.isoformat(), "budgets": projected,
```

```
                "adjusted_by_bounds": {s: d for s, d in changed.items() if abs(d) > 1e-6}}, 4)
```

Code 29: Budgets to target weights through the C++ engine
python/nabia/ai/servers/portfolio.py:82-93

```
def compute_weights(ctx: ServerContext, as_of: dt.date, budgets: dict[str, float]) -> dict:
    """Runs nabia.latest_weights on prices up to as_of and stores the weights with the decision."""
    import nabia  # the C++ extension, imported on use

    days = int(ctx.config.portfolio.get("covariance_window", 60)) * 3 + 30
    prices = ctx.prices(as_of, days=max(days, 120))
    target = nabia.latest_weights(prices, {**ctx.config.portfolio, "budgets": budgets})
    weights = {s: float(w) for s, w in target.weights.items()}
    ctx.store.save_weights(ctx.run, as_of, weights, target.leverage)
    return clean({"as_of": as_of.isoformat(), "priced_on": target.date.date().isoformat(),
                  "weights": weights, "risk_contributions": target.risk_contributions.to_dict(),
                  "leverage": target.leverage, "converged": target.converged}, 4)
```

12.6 The trend rule baseline

A transparent, model-free policy serves as the benchmark that agent policies must beat:

$$b_i \;\propto\; 1 + s \cdot \mathrm{clip}(m_i, -1, 1), \qquad s = 0.5,$$

followed by the same bounded normalization. A strong uptrend gets up to 1.5× the equal budget and a strong downtrend 0.5×. Over 2019–2026 on the default universe with monthly decisions it reached a Sharpe ratio of 0.96, against 0.92 for equal risk.

Code 30: The trend rule
python/nabia/ai/analytics.py:94-104

```
def trend_tilted_budgets(prices: pd.DataFrame, strength: float = 0.5, min_budget: float = 0.02,
                         max_budget: float = 0.40) -> dict[str, float]:
    """A transparent rule-based policy: equal risk budgets tilted towards assets
    with positive volatility-scaled momentum, bounded and normalized.

    budget_i ~ 1 + strength * clip(momentum_12_1_i, -1, 1). Used as a baseline
    for the agents and as a deterministic decision rule in tests.
    """
    momentum = trend_signals(prices)["momentum_12_1"].fillna(0.0).clip(-1.0, 1.0)
    raw = (1.0 + strength * momentum).clip(lower=1e-6)
    return project_budgets(raw.to_dict(), min_budget, max_budget)
```

12.7 Backtest mode

At each decision date (every trading day, week, month or quarter end), the policy decides budgets as of that date. Policies are the trend rule, the agents on the local model, the agents on Claude, or the agents on the configured provider. The decisions form a budget schedule that drives one continuous backtest, compared with equal risk budgets over the same period. Decisions are stored per run, so interrupted backtests resume.

Code 31: Decision dates
python/nabia/ai/pipeline.py:52-60

```
def decision_dates(index: pd.DatetimeIndex, start: dt.date, end: dt.date, frequency: str = "ME") ->
     list[dt.date]:
    """Decision dates within [start, end]: every trading date for "D" (daily),
    otherwise the last trading date of each period (pandas offset alias: "ME"
    month end, "W-FRI" weekly, "QE" quarter end)."""
    dates = pd.Series(index, index=index)
    dates = dates[(dates >= pd.Timestamp(start)) & (dates <= pd.Timestamp(end))]
    if frequency.upper() in ("D", "B"):
        return [d.date() for d in dates]
    return [d.date() for d in dates.groupby(pd.Grouper(freq=frequency)).max().dropna()]
```

Code 32: Decisions to a continuous backtest, against equal risk
`python/nabia/ai/pipeline.py:176-192`

```
def evaluate_decisions(config: AIConfig, run: str, start: dt.date, end: dt.date,
                       prices: pd.DataFrame | None = None) -> AgentBacktest:
    """Backtest of the decisions already stored for ‘run‘ between start and end
    (no new decisions), against equal risk budgets."""
    import nabia

    prices = _backtest_prices(config, start, end) if prices is None else prices
    decisions = [d for d in AgentStore(config.db_url).decisions(run)
                 if start.isoformat() <= d["as_of"] <= end.isoformat()]
    if not decisions:
        raise ValueError(f"run {run!r} has no decisions between {start} and {end}")
    schedule = pd.DataFrame([d["budgets"] for d in decisions],
                            index=pd.DatetimeIndex([d["as_of"] for d in decisions], name="date"))[
     config.symbols]
    params = dict(config.portfolio)
    result = nabia.backtest(prices, params, budget_schedule=schedule)
    baseline = nabia.backtest(prices, params)
    return AgentBacktest(result, baseline, schedule, decisions)
```

Look-ahead control. The tools cannot see the future. In backtests, news comes only from the database (no web access), and the prompt instructs the model to treat the decision date as the present. One leak cannot be closed by tooling: a language model was trained on data up to its knowledge cutoff and may recognize market history before it. Agent backtests inside that window are therefore optimistic. They are fully out-of-sample only after the cutoff.

Code 33: The as-of rule given to every agent
`python/nabia/ai/agents/team.py:17-20`

```
AS_OF_RULE = (
    "Today is {as_of}. Treat that date as the present: base every judgement on what the tools return "
    "(they only see data up to {as_of}) and never on knowledge of events after {as_of}."
)
```

13 News: collection, sentiment and relevance

13.1 Collection

The news analyzer runs once per day in live mode:

1. It calls `query_news` to see what is already stored, so nothing is stored twice.
2. It gathers candidate stories:
   - `fetch_news_feeds` reads public RSS and Atom feeds (MarketWatch, CNBC, Financial Times, Investing.com, CoinDesk, OilPrice, the Federal Reserve, the ECB and the Bank of England; configurable). It keeps items published within the last d days and removes URLs already in the database. A broken feed is reported without hiding the others.
   - On Claude, web search and fetch also fill gaps and confirm stories.
3. It keeps the 8–20 most material stories, classifies each, and stores them with `store_news_articles`, keyed by URL.

Code 34: RSS and Atom headlines, without already-stored articles
`python/nabia/ai/servers/news.py:71-90`

```
@tool(server)
def fetch_news_feeds(max_age_days: int = 2, max_items: int = 60) -> dict:
```

```
    """Latest headlines from the configured financial news RSS feeds (markets,
    economy, central banks, energy, crypto), newest first, skipping articles
    already stored. Use it to find candidate stories to classify and store."""
    cutoff = dt.datetime.now(dt.UTC) - dt.timedelta(days=max_age_days)
    items, errors = [], []
    for source, url in ctx.config.news_feeds.items():
        try:
            items += [i for i in _read_feed(source, url) if i["published"] is None or i["published"] >=
      cutoff]
        except Exception as error:  # one broken feed must not hide the others
            errors.append(f"{source}: {type(error).__name__}")
    known = ctx.store.known_urls([i["url"] for i in items])
    fresh, seen = [], set()
    for i in sorted(items, key=lambda i: i["published"] or cutoff, reverse=True):
        if i["url"] in known or i["url"] in seen:
            continue
        seen.add(i["url"])
        fresh.append({**i, "published": i["published"].date().isoformat() if i["published"] else None})
    return {"items": fresh[:max_items], "already_stored": len(known), "feed_errors": errors}
```

13.2 How sentiment and relevance are produced

Sentiment and relevance are judgements made by the language model for each article. They are not computed by a statistical model or a word list. The news analyzer reads the headline and description (and, on Claude, possibly the article) and assigns the fields of the `NewsArticle` schema following the rubric in its instructions:

Sentiment $s \in [-1, 1]$: the *expected price impact on the affected assets*: -1 clearly bearish, $+1$ clearly bullish, 0 neutral or mixed. It is defined relative to the assets the story moves, not as the tone of the text. A hawkish central bank is bearish for bonds even when the article sounds upbeat.

Relevance $r \in [0, 1]$: the importance of the story for a cross-asset portfolio: 1 for portfolio-wide shocks, about 0.5 for stories that move one asset class, lower for minor items.

Classification a topic from a fixed list (monetary policy, inflation, growth, labor, earnings, fiscal, geopolitics, energy, metals, agriculture, crypto, rates, credit, FX, regulation, other), the affected asset classes, and the portfolio symbols directly affected.

Code 35: The news analyzer's instructions, which define the rubric
`python/nabia/ai/agents/team.py:22-43`

```
NEWS_SYSTEM = """You are the news analyst of a systematic cross-asset trend portfolio.

{as_of_rule}

Portfolio universe (symbol: name, asset class):
{universe}

Your job: find the financial news published on {as_of} and the previous day that matters for these
assets, and store it in the news database. Cover central banks and rates, inflation and growth data,
equity markets, energy and metals, and crypto; prefer primary and reputable sources.

How to work:
1. Call query_news for {as_of} (days=2) to see what is already stored; do not store it again.
2. {research}
   Aim for 8-20 distinct, material articles, not every headline: skip stories with no plausible
   effect on these assets.
3. Store them with store_news_articles (in batches is fine). Use only URLs and dates you actually saw
   in the feed items or search results; never invent them. topic: the main driver. asset_classes and
     symbols: what the story
   moves (symbols only from the universe above). sentiment: expected price impact on the affected
   assets, -1 clearly bearish to +1 clearly bullish (0 neutral or mixed). relevance: 1 for
   portfolio-wide shocks, 0.5 for one asset class, lower for minor items.
4. Finish with five lines summarizing the dominant themes and their direction."""
```

The server validates what the model produces before storing it:

- the schema bounds ($s \in [-1, 1]$, $r \in [0, 1]$, a topic from the list, valid asset classes);
- no publication dates in the future;
- symbols must belong to the portfolio universe.

Invalid articles are rejected with a reason the model can act on.

Code 36: The article schema: bounds and meaning of each field
`python/nabia/ai/servers/news.py:27-39`

```python
class NewsArticle(BaseModel):
    """One news item, as extracted by the news analyzer."""

    url: str = Field(description="Canonical URL of the article (the unique key)")
    published_date: dt.date = Field(description="Publication date, YYYY-MM-DD")
    title: str
    source: str = Field(description="Publisher, e.g. Reuters, Bloomberg, FT")
    topic: Topic = Field(description="Main topic")
    asset_classes: list[AssetClass] = Field(description="Asset classes the news affects")
    symbols: list[str] = Field(default_factory=list, description="Portfolio symbols directly affected")
    sentiment: float = Field(ge=-1.0, le=1.0, description="Price impact on the affected assets: -1
     bearish .. +1 bullish")
    relevance: float = Field(ge=0.0, le=1.0, description="Importance for a cross-asset portfolio: 0 ..
     1")
    summary: str = Field(description="Two or three factual sentences: what happened and why it matters"
    )
```

Code 37: Validation before storage
`python/nabia/ai/servers/news.py:50-68`

```python
@tool(server)
def store_news_articles(articles: list[NewsArticle]) -> dict:
    """Store news articles (upsert by URL, so storing an article twice updates it).

    Only store articles with a real URL and publication date you have seen.
    Symbols must be portfolio symbols (see the news_summary universe)."""
    today = dt.date.today()
    universe = set(ctx.config.symbols)
    rejected = []
    accepted = []
    for a in articles:
        if a.published_date > today:
            rejected.append({"url": a.url, "reason": "published_date is in the future"})
        elif not set(a.symbols) <= universe:
            rejected.append({"url": a.url, "reason": f"unknown symbols {sorted(set(a.symbols) -
     universe)}"})
        else:
            accepted.append(a.model_dump())
    stored = ctx.store.store_news(accepted)
    return {"stored": stored, "rejected": rejected}
```

13.3 Aggregation

Agents and the web app consume aggregates. For the articles A published in the d days up to and including `as_of`, `news_summary` groups articles by topic, asset class and symbol. An article that affects several asset classes or symbols counts in each of their groups. For each group g it reports the count, the maximum relevance, and the **relevance-weighted sentiment**

$$S_g = \frac{\sum_{i \in g} s_i \, r_i}{\sum_{i \in g} r_i},$$

with the denominator replaced by 1 when all relevances are zero (so $S_g = 0$). Weighting by relevance lets one portfolio-wide story outweigh several minor ones in the same group. The

summary also lists the ten most relevant headlines. The web app's *News* page shows article counts per day and the simple (unweighted) average sentiment per topic over the selected filter.

Code 38: Relevance-weighted aggregation
`python/nabia/ai/servers/news.py:101-134`

```python
@tool(server)
def news_summary(as_of: str, days: int = 7) -> dict:
    """Aggregate view of the news in the `days` days up to as_of: article counts and
    relevance-weighted sentiment per topic, asset class and symbol, plus the top headlines."""
    end = parse_as_of(as_of)
    articles = ctx.store.news(end - dt.timedelta(days=days - 1), end, limit=10_000)

    def aggregate(key_of) -> dict:
        groups: dict[str, list[dict]] = {}
        for a in articles:
            for key in key_of(a):
                groups.setdefault(key, []).append(a)
        out = {}
        for key, items in sorted(groups.items()):
            weight = sum(i["relevance"] for i in items) or 1.0
            out[key] = {
                "articles": len(items),
                "weighted_sentiment": sum(i["sentiment"] * i["relevance"] for i in items) / weight,
                "max_relevance": max(i["relevance"] for i in items),
            }
        return out

    top = sorted(articles, key=lambda a: a["relevance"], reverse=True)[:10]
    return clean({
        "as_of": end.isoformat(),
        "days": days,
        "universe": ctx.config.symbols,
        "articles": len(articles),
        "by_topic": aggregate(lambda a: [a["topic"]]),
        "by_asset_class": aggregate(lambda a: a["asset_classes"]),
        "by_symbol": aggregate(lambda a: a["symbols"]),
        "top_headlines": [{k: a[k] for k in ("published_date", "title", "source", "topic", "symbols",
                                             "sentiment", "relevance", "summary")} for a in top],
    }, 3)
```

13.4 How the execution agent uses news, and limitations

News is a *secondary* input by design. The execution agent starts from equal risk, tilts mainly on the statistical evidence, and uses news to confirm, temper or flag risks the statistics do not show yet. Because the scores are model judgements:

- **They are not reproducible across models**, and only approximately across runs (the local model runs at temperature 0.2). Decisions record the model that made them so runs can be compared.
- **They are uncalibrated**: a sentiment of 0.6 has no defined relation to a return. Using them as quantitative signals would first require calibration against subsequent price moves, for example regressing next-day asset returns on S_g.
- **Coverage depends on the sources**: RSS headlines and descriptions are short. Claude's web access gives fuller context than the local setup.

Possible improvements are an ensemble of scoring models, a lexicon or finance-tuned classifier as a reproducible baseline, and a calibration study of S_g against returns.

14 Web application and job runner

`nabia.web` is a FastAPI application with a single-page JavaScript front end (Chart.js).

Overview the latest decision, market data coverage, recent jobs.

News filters, counts per day, sentiment by topic, article detail.

Executions budgets over time per run, each decision's rationale and statistics report, and every agent run's tool calls and token usage.

Backtests launch runs with a policy and a decision frequency, then compare them with equal risk (equity, drawdown, metrics, budgets, weights).

Jobs market data downloads, news collection, daily decisions, backtests.

Jobs run one at a time in the server's event loop. SQLite allows one writer, and agent runs are costly. Their status is stored in the database.

Job logs. A logging handler stores every record emitted while a job runs under that job's id. The job id lives in a *context variable*, which follows the job into its asyncio tasks and its `asyncio.to_thread` workers (downloads), so concurrent web requests never leak into a job's log. Chatty libraries contribute only warnings and errors. The front end tails the log incrementally (`GET /api/jobs/{id}/logs?after=`).

Code 39: Per-job log capture
`python/nabia/web/jobs.py:36-53`

```python
class JobLogHandler(logging.Handler):
    """Stores the log records emitted while a job runs, under that job's id."""

    def __init__(self, store: AgentStore) -> None:
        super().__init__(logging.INFO)
        self.store = store
        self.setFormatter(logging.Formatter("%(message)s"))

    def emit(self, record: logging.LogRecord) -> None:
        job_id = current_job.get()
        if job_id is None:
            return
        if record.name.startswith(_QUIET) and record.levelno < logging.WARNING:
            return
        try:
            self.store.add_job_log(job_id, record.levelname, record.name, self.format(record))
        except Exception:  # logging must never break the job
            self.handleError(record)
```

Code 40: Running a job with its log context
`python/nabia/web/jobs.py:105-119`

```python
async def _run(self, job_id: int, kind: str, params: dict[str, Any]) -> None:
    async with self._lock:
        token = current_job.set(job_id)
        try:
            self.store.update_job(job_id, status="running", message="started")
            log.info("job #%s: %s %s", job_id, kind, params or "")
            message = await getattr(self, f"_{kind}")(job_id, params)
            log.info("job #%s done: %s", job_id, message)
            self.store.update_job(job_id, status="done", message=message)
        except Exception as error:
            log.exception("job #%s failed", job_id)  # with the traceback, in the job log
            self.store.update_job(job_id, status="failed", error="".join(
                traceback.format_exception_only(error)).strip())
        finally:
            current_job.reset(token)
```

15 Verification

- **C++** (Catch2): 88 test cases. They check results against closed forms (inverse-volatility weights for uncorrelated assets, Gaussian VaR and ES quantiles, SES and Kalman steady states) and accounting identities to 10^{-15}. They also check no look-ahead, thread-count independence, and invalid input. The suites run in debug and under AddressSanitizer with UndefinedBehaviorSanitizer, and the multithreaded code under ThreadSanitizer.
- **Python** (pytest): 37 tests, offline by default:
  - the analytics, and the three MCP servers in process (including the no-look-ahead check);
  - the agent loop over real stdio MCP servers with a scripted model client (parallel calls, tool errors, paused turns, refusals);
  - the Ollama translation against a mock transport, RSS parsing;
  - the pipeline and the web API, including job logs.

  Opt-in tests download live market data and run the execution agent end to end on a local model.
- **Documents**: this document's code excerpts are extracted from the source at build time and cite their file and lines. A listed function that no longer exists stops the build instead of leaving a stale excerpt.